

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

August 17, 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 1: Fundamentos de paralelismo y arquitecturas paralelas

Al finalizar la unidad, los alumnos conocen:

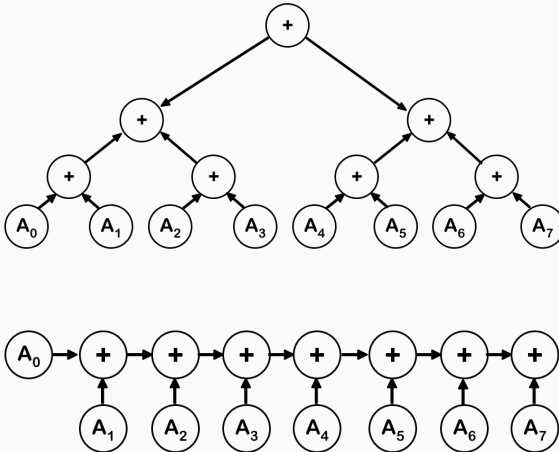
1. Modelo secuencial (y limitaciones)
2. Paralelismo y arquitecturas paralelas (paradigmas)
3. Modelos escalables (optimización en paralelo)
4. Taxonomía de Flynn
5. Diseño en paralelo: método de Foster
6. **DAG (Directed Acyclic Graphs)**

U1.6. DAG

(Directed Acyclic Graphs)

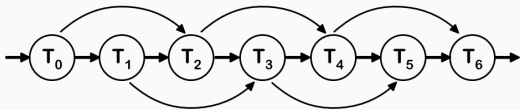
Modelo DAG (Directed Acyclic Graph)

- vértices representan **tareas** (instrucciones simples o en bloques)
- aristas representan **canales de comunicación** (dependencias)

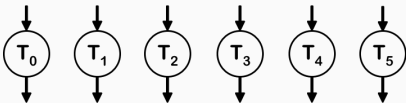


Podemos clasificar a los algoritmos de la siguiente forma:

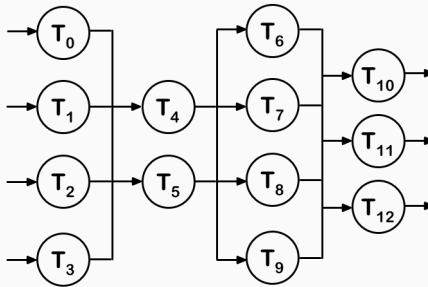
- **Secuenciales**, no pueden ser paralelizados porque todas las tareas tienen dependencias en tareas previas



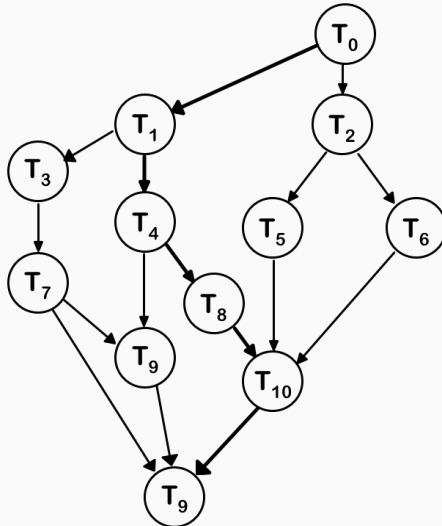
- **Paralelos**, donde todas las tareas pueden ser ejecutadas simultáneamente



- **SPA** (Secuencial-Paralelo) El algoritmo está separado en niveles, que se ejecutan en paralelo, pero los cuales tienen una forma secuencial de ejecución.



- **NSPA** (No-Secuencial-Paralelo) El algoritmo no contiene ninguno de los patrones anteriores



Modelo de costo DAG

El **scheduling** o 'plan de ejecución' de un DAG (V, E) , asigna a cada nodo i :

- un **tiempo de ejecución** t_i , tal que si $(i, i + 1) \in E$, el tiempo acumulado $T_{i+1} = T_i + t_{i+1}$. Si las tareas son $O(1)$, $T_{i+1} = T_i + 1$

- un **procesador** p_i , $i \in \{0, \dots, p - 1\}$

(los nodos de entrada, en ciertos casos, corresponden a $t_0 = 0$)

El algoritmo paralelo correspondiente se ejecuta en

$$T_{\infty} = \text{MAX}(\sum t_i)$$

que es la longitud del plan de ejecución, llamada **span**

El **trabajo** (W) se define como la cantidad total de operaciones (instrucciones). Es decir, la complejidad secuencial (T_s)

Se busca que el DAG tenga un **trabajo** (mínimo) y un **span** (profundidad) mínima

El **trabajo** equivale al tiempo secuencial (optimo): $W(n) = T_s^*(n)$

En paralelo, p procesos ejecutaran por lo menos T_s/p unidades de tiempo. Es decir

$$T_p \geq T_s/p$$

Esto nos permite definir el **speedup** (velocidad) de un proceso en paralelo como:

$$S = \frac{T_s}{T_p}$$

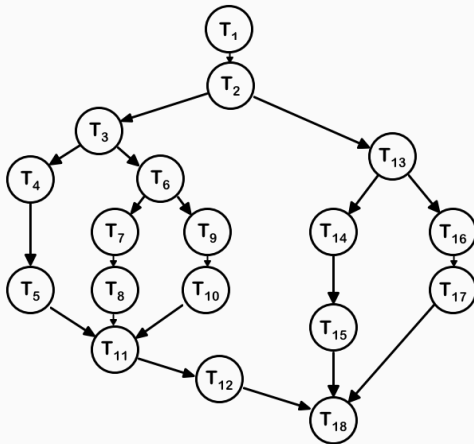
Un speedup igual (o proporcional) a p es un speedup lineal optimo, ya que $S = \frac{T_s}{T_p} \leq p$

Para el **span**, o camino critico del DAG, tenemos que

$$T_p \geq T_\infty$$

Ejemplo 1

Determine span y trabajo en el siguiente DAG si cada tarea tiene complejidad constante, $O(1)$



Ejemplo 2: función recursiva

Considere la función :

$$f(x) = x, x \leq 1$$

$$f(x) = f(x-1) + f(x-2), x > 1$$

$$\text{i.e. } (a,b) = (f(x-1) \parallel f(x-2))$$

Determine el span y trabajo

Ejemplo 3: Suma de números

- a) Construya un DAG eficiente para la suma de n enteros en paralelo, si se tiene $n=16$ y $p=8$ (procesadores disponibles)
- b) Compare el DAG con el caso $n=16$, $p=5$
- c) Determine T_s , $T_\infty(n)$, $W(n)$, $S(n)$ y la clasificación del algoritmo según Flynn en cada caso

Ejemplo 4

Diagrame el DAG correspondiente al siguiente código

```
double a[N], b[N], c[N], v=0.0, w=0.0;  
T1(a, &v);  
T2(b, &w);  
T3(b, &v);  
T4(c, &w);  
T5(c, &v);  
T6(a, &w);
```

Las funciones leen y modifican ambos argumentos.

Determine T_s , $T_\infty(n)$, $W(n)$, $S(n)$ del algoritmo paralelo asumiendo una complejidad constante de las tareas dadas.

Ahora recalcule T_s , $T_\infty(n)$, $W(n)$, $S(n)$ si la complejidades algorítmicas son: $T_1(3n)$, $T_2(5n)$, $T_5(n)$, $T_6(2n)$ mientras que $T_3(n) = 2n \log n$, $T_4(n) = 3 \log n$

Discuta sobre la cantidad de procesos necesaria para resolver el problema



Joseph Jája *An introduction of parallel algorithms*. University of Maryland



Fayez Gebali *Algorithms and Parallel Computing* Wiley, 2011



Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.

